DuocUC® INFORMÁTICA Y TELECOMUNICACIONES

Fundamentos de HTTP y REST: Conceptos y Buenas Prácticas

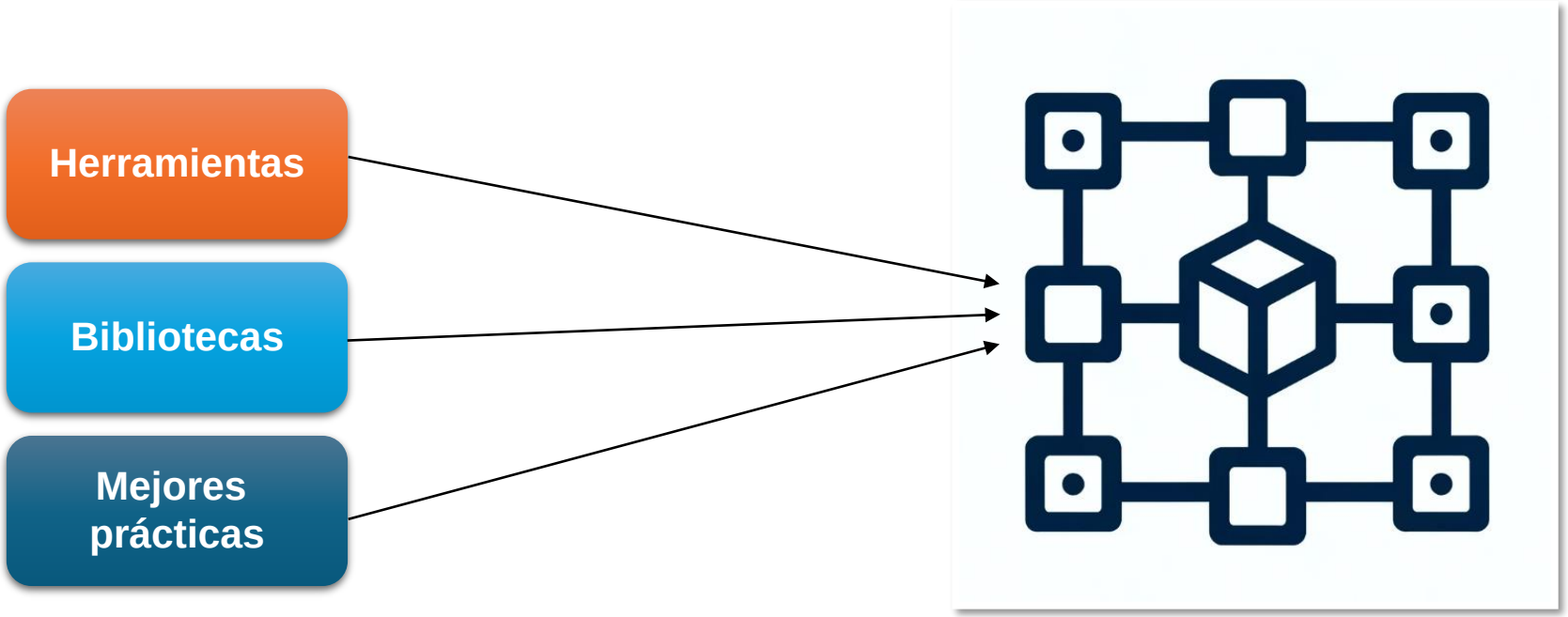
DESARROLLO FULLSTACK I

DSY1103

FORMAMOS PERONAS PARA UNA SOCIEDAD MEJOR

En el capítulo anterior…

¿Qué diferencia hay entre Framework y Biblioteca?



ESCUELA DE INFORMÁTICA Y TELECOMUNICACIONES

DuocUC®

• Contenidos

- ¿Qué es la Web y cómo funciona?.
- Conceptos de HTTP.
- Métodos y Respuestas de HTTP.
- REST.
- ¿Qué son las APIs?.
- Buenas prácticas en APIs REST.
- POSTMAN.
- Sintaxis Básica Spring Boot.
- Actividad.

*“La tecnología **no es una cosa**, es una forma de **hacer cosas**”*

– Tim O’Reilly

01

Cómo funciona la Web

¿Qué es la **Web**?

• ¿Qué es la Web?

La **Web**, o **World Wide Web**, es una red global de información que permite el acceso a documentos y recursos a través de Internet. Consiste en:

PÁGINAS WEB

Documentos de hipertexto que contienen texto, imágenes, videos y enlaces.

SITIOS WEB

Colecciones de páginas web relacionadas, generalmente alojadas en un mismo dominio.

SERVIDORES WEB

Computadoras que almacenan y entregan páginas web a los usuarios.

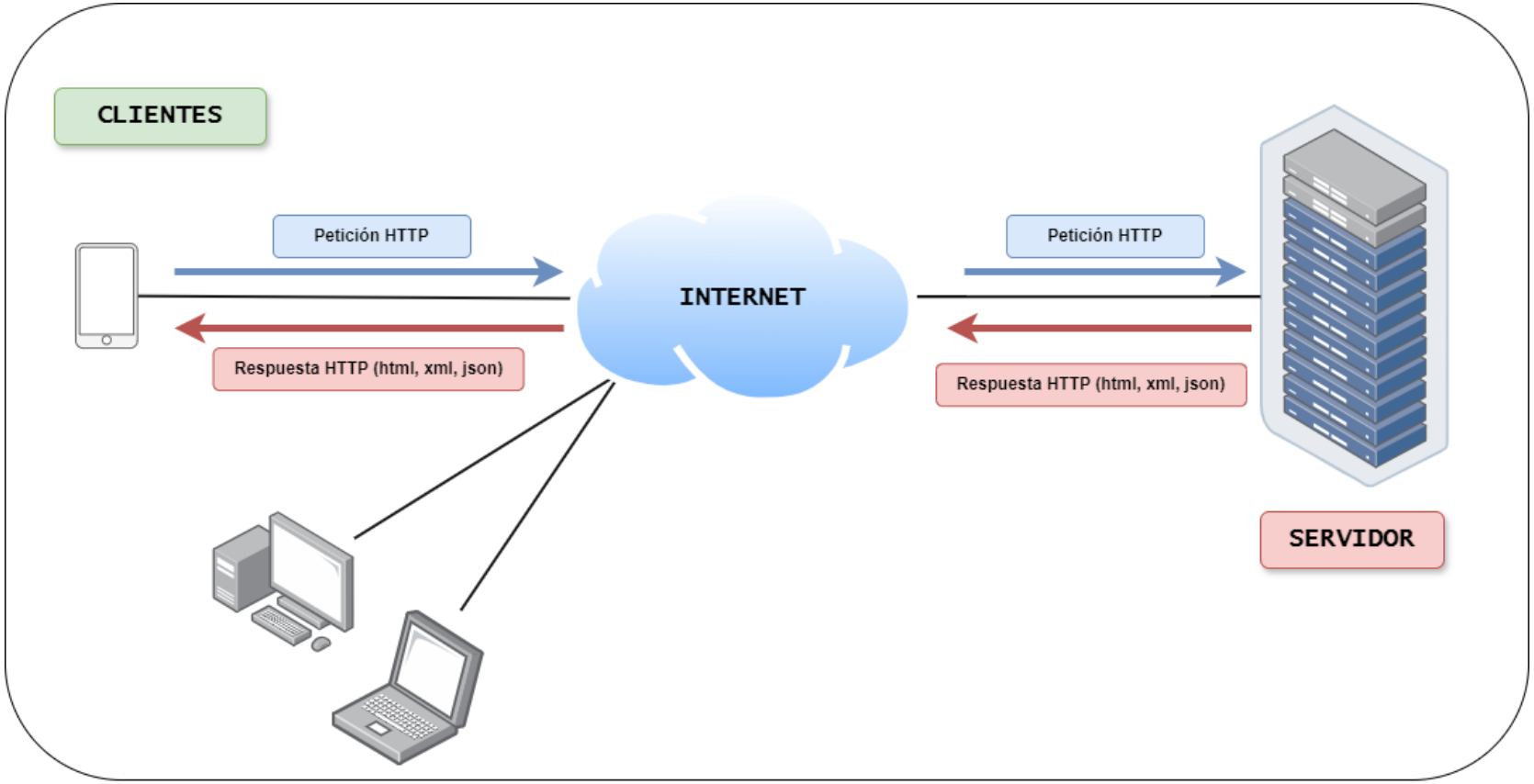
NAVEGADORES WEB

Programas que permiten a los usuarios ver e interactuar con páginas web.

¿Cómo funciona la **Web**?

• ¿Cómo funciona la Web?

La web funciona con un modelo **cliente-servidor**. El cliente (navegador web) envía solicitudes al servidor, y el servidor responde con el contenido solicitado.



¿Qué se entiende por HTML?

index.html

A > index.html > html > body

```html
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>💪 MATHIAS STORE 💪 - Tienda de Productos Deportivos 🏋️ </title>
    <link rel="stylesheet" href="styles.css">
</head>
<body>
    <header>
        <div class="container">
            <h1>💪 MATHIAS STORE 🏋️ </h1>
            <nav>
                <ul>
                    <li><a href="#home">Inicio</a></li>
                    <li><a href="#products">Productos</a></li>
                </ul>
            </nav>
        </div>
    </header>
    <main>
        <section id="home">
            <div class="container">
                <h2>💪 Bienvenido a MATHIAS STORE 🏋️ </h2>
                <p>Tu tienda online de productos deportivos de alta calidad.</p>
            </div>
        </section>
        <section id="products">···
        </section>
        <section id="about">···
        </section>
        <section id="contact">···
        </section>
    </main>
    <footer>···
    </footer>
</body>
</html>
```

• HTML

HTML es un lenguaje de marcado utilizado para crear y estructurar páginas web.

Está diseñado principalmente para la **presentación** y **navegación** de contenido en la web.

HTML utiliza principalmente los métodos **HTTP GET** y **POST** para interactuar con los servidores.

GET se usa para solicitar y mostrar contenido.

POST se usa para enviar datos de formularios al servidor.

Tradicionalmente, **HTML** solo soporta métodos **GET** y **POST** para formularios.

Los métodos **PUT, PATCH** y **DELETE** no son nativamente compatibles con formularios HTML.

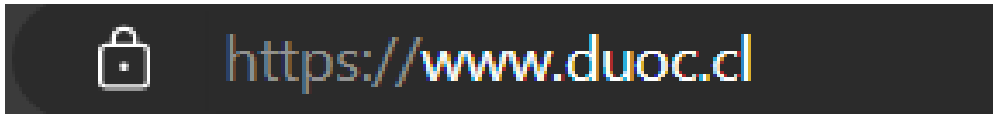
02

Conceptos de HTTP

• ¿Qué es HTTP?

HTTP **(HyperText Transfer Protocol)** es el protocolo utilizado por la web para la transferencia de documentos y datos.

El usuario ingresa una URL en el navegador.



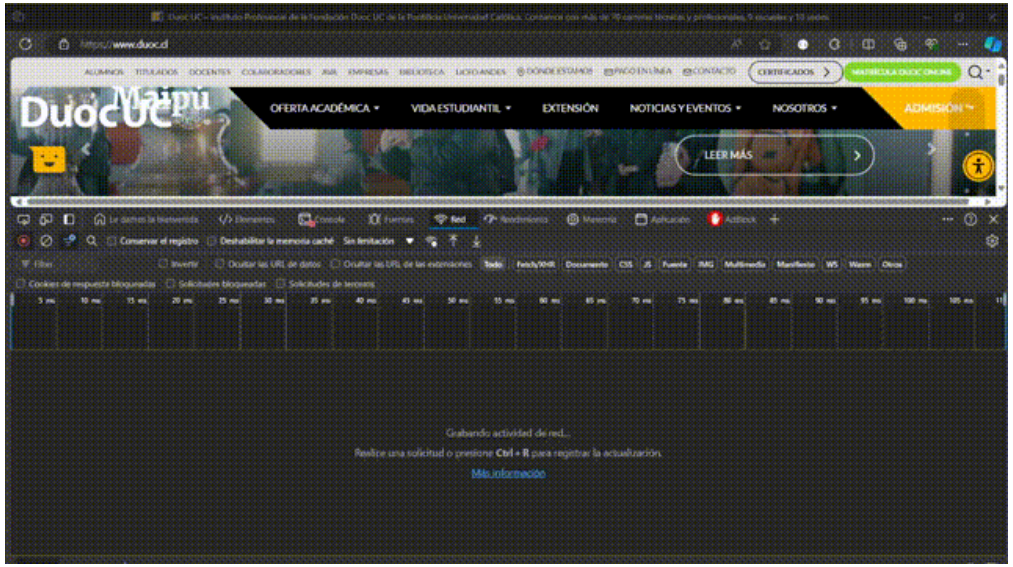
El navegador envía una solicitud HTTP al servidor web correspondiente.

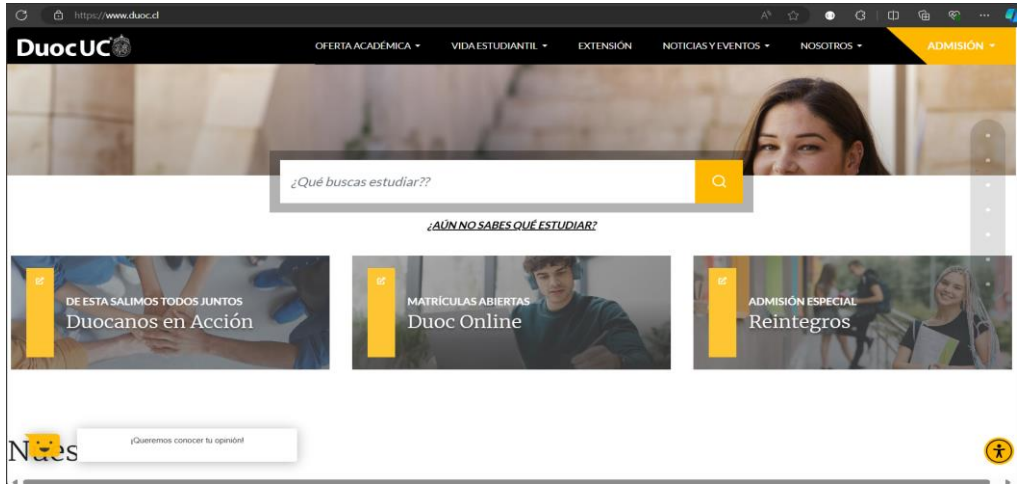
• ¿Qué es HTTP?

El servidor web responde con el archivo HTML de la página web.

El navegador analiza el HTML y descarga los recursos adicionales (CSS, JavaScript, imágenes, etc.).

El navegador renderiza la página web completa para el usuario.





• Métodos de HTTP

Son las **acciones** que se pueden realizar sobre los recursos en una arquitectura web, como solicitar, crear, actualizar o eliminar información. Cada uno tiene un propósito y semántica específica para interactuar con los recursos en el servidor.

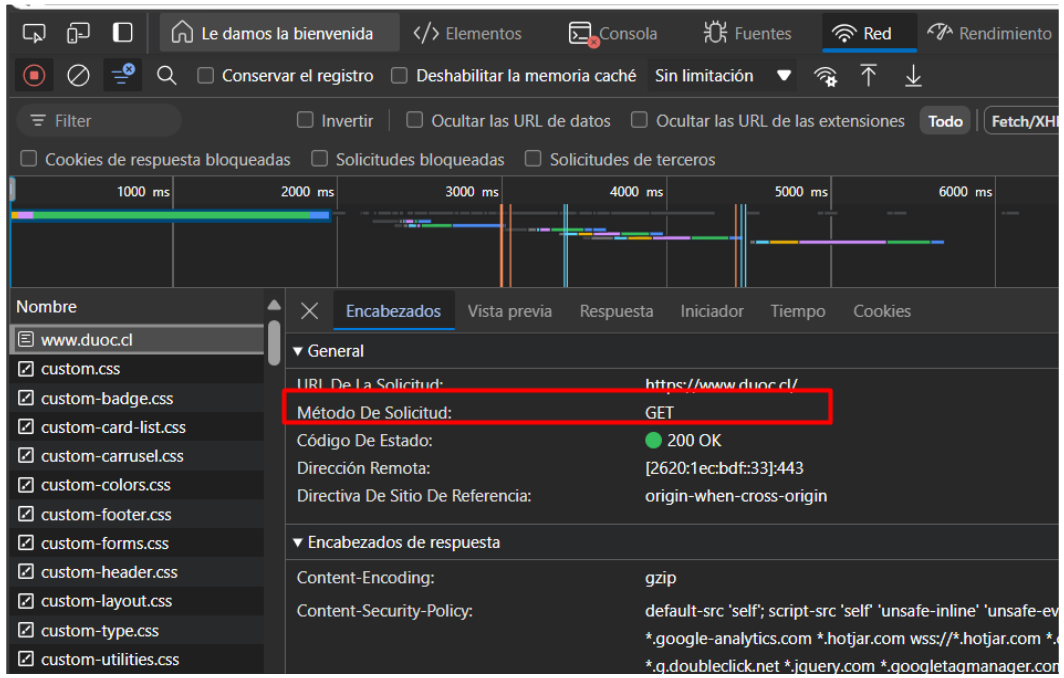
- GET
- POST
- PUT
- DELETE
- HEAD
- OPTIONS
- PATCH

• Método GET

GET

El método **GET** se utiliza para **solicitar datos de un servidor.** Es una operación de solo lectura y no modifica los datos en el servidor.

Siempre que navegamos por internet o en una aplicación utilizamos este método para obtener recursos.



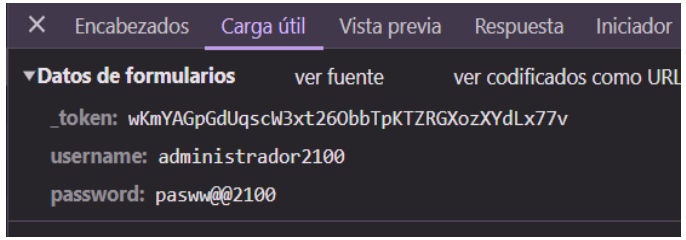
• Método POST

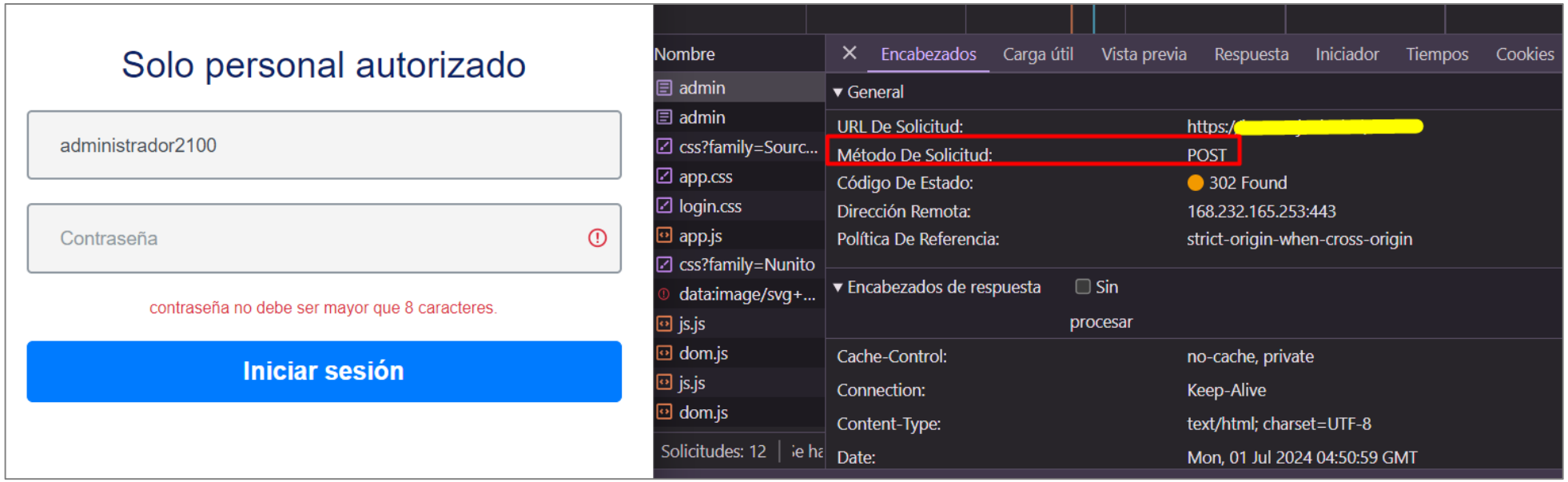
POST

El método **POST** se utiliza para **enviar datos al servidor** con el fin de crear un nuevo recurso.

Se utiliza para enviar formularios, archivos, etc.

Los datos enviados se incluyen en el cuerpo de la solicitud.





• Método PUT

PUT

El método **PUT** se utiliza para **reemplazar por completo un recurso existente** en el servidor con un nuevo conjunto de datos. Cuando se envía una solicitud **PUT**, se deben incluir todos los campos del recurso en el cuerpo de la solicitud, incluso si algunos de ellos no han cambiado.

• Método PATCH

PATCH

El método **PATCH** se utiliza para realizar **una actualización parcial de un recurso existente** en el servidor. Cuando se envía una solicitud PATCH, solo se incluyen los campos del recurso que se van a actualizar en el cuerpo de la solicitud. Los campos que no se incluyen en la solicitud PATCH no se modifican.

PUT reemplaza por completo un recurso, mientras que PATCH actualiza solo los campos especificados de un recurso existente. PATCH es más eficiente cuando solo se necesita actualizar una parte de un recurso, ya que reduce el tamaño de la solicitud y la carga de procesamiento en el servidor.

• Método DELETE

DELETE

El método **DELETE** se utiliza para **eliminar un recurso del servidor**, no se incluyen datos en el cuerpo de la solicitud.

• Método HEAD

HEAD

El método **HEAD** es similar al método **GET**, pero en lugar de **devolver el cuerpo de la respuesta**, **solo devuelve los encabezados de la respuesta**.

OPTIONS

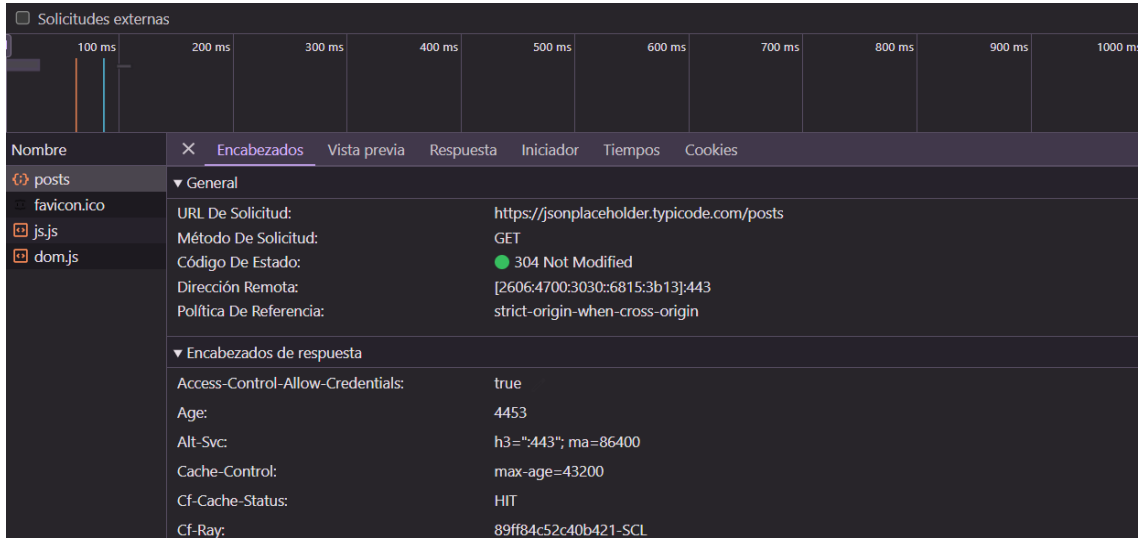
• Método OPTIONS

El método **OPTIONS** se utiliza para **solicitar información sobre las opciones de comunicación disponibles** para un recurso determinado o un servidor en general.

• Respuestas en HTTP

Son **mensajes enviados por el servidor al cliente** (normalmente un navegador web o una aplicación) en respuesta a una solicitud **HTTP**. Estas respuestas indican si la solicitud fue **exitosa** y proporcionan los datos solicitados o una explicación del error si la solicitud no se pudo completar.

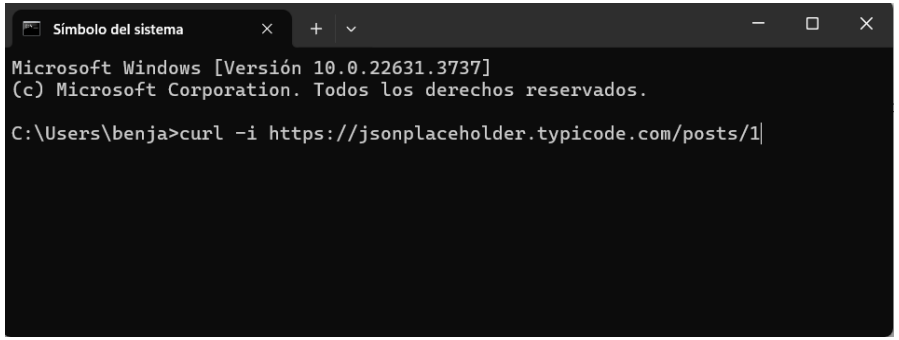
Cada respuesta HTTP incluye un código de estado HTTP y, opcionalmente, datos adicionales en el cuerpo de la respuesta.

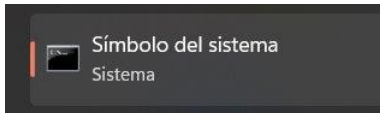


• Estructura de una Respuesta HTTP

Una respuesta HTTP típica tiene la siguiente estructura:

- **Línea de estado:** Indica el resultado de la solicitud, incluido el código de estado HTTP y una breve descripción.
- **Encabezados HTTP:** Proporcionan información adicional sobre la respuesta, como el tipo de contenido, la longitud del contenido y las políticas de caché.
- **Cuerpo de la respuesta:** Contiene los datos solicitados, como una página HTML, un archivo JSON, una imagen, etc.





1.- Abrir **Símbolo del sistema**
2.- escribir el siguiente comando curl

curl -i https://jsonplaceholder.typicode.com/posts/1

3.- Observa
4.- Identifica el **estado, encabezados y el cuerpo.**

• Códigos de Estado HTTP

1XX

CÓDIGOS INFORMATIVOS

El servidor ha recibido la petición y procederá con ella.

2XX

CÓDIGOS DE ÉXITO

El servidor ha recibido, entendido y procesado la solicitud correctamente.

3XX

CÓDIGOS DE REDIRECCIÓN

Estos códigos indican que se deben tomar más acciones para completar la solicitud, generalmente redirigiendo a una URL diferente..

4XX

CÓDIGOS DE ERROR DE CLIENTE

Estos códigos indican que hubo un problema con la solicitud del cliente.

5XX

CÓDIGOS DE ERROR DE SERVIDOR

Estos códigos indican que el servidor falló al cumplir una solicitud válida.

- **100 Continue:** El cliente debe continuar con la solicitud.
- **101 Switching Protocols:** El servidor acepta cambiar a un protocolo diferente.

- **200 OK:** La solicitud ha tenido éxito.
- **201 Created:** La solicitud ha sido cumplida y se ha creado un nuevo recurso.
- **202 Accepted:** La solicitud ha sido aceptada para procesamiento, pero el procesamiento no se ha completado.
- **204 No Content:** La solicitud ha tenido éxito, pero no hay contenido que enviar en la respuesta.

• Códigos de Estado HTTP

1XX

CÓDIGOS INFORMATIVOS

El servidor ha recibido la petición y procederá con ella.

2XX

CÓDIGOS DE ÉXITO

El servidor ha recibido, entendido y procesado la solicitud correctamente.

3XX

CÓDIGOS DE REDIRECCIÓN

Estos códigos indican que se deben tomar más acciones para completar la solicitud, generalmente redirigiendo a una URL diferente..

4XX

CÓDIGOS DE ERROR DE CLIENTE

Estos códigos indican que hubo un problema con la solicitud del cliente.

5XX

CÓDIGOS DE ERROR DE SERVIDOR

Estos códigos indican que el servidor falló al cumplir una solicitud válida.

- **301 Moved Permanently:** El recurso solicitado ha sido movido permanentemente a una nueva URL.
- **302 Found:** El recurso solicitado se encuentra temporalmente en una URL diferente.
- **304 Not Modified:** El recurso no ha sido modificado desde la última solicitud.

- **400 Bad Request:** La solicitud no pudo ser entendida o estaba mal formada.
- **401 Unauthorized:** La autenticación es necesaria y ha fallado o no se ha proporcionado.
- **403 Forbidden:** El servidor entiende la solicitud, pero se niega a autorizarla.
- **404 Not Found:** El recurso solicitado no pudo ser encontrado en el servidor.

• Códigos de Estado HTTP

1XX

CÓDIGOS INFORMATIVOS

El servidor ha recibido la petición y procederá con ella.

2XX

CÓDIGOS DE ÉXITO

El servidor ha recibido, entendido y procesado la solicitud correctamente.

3XX

CÓDIGOS DE REDIRECCIÓN

Estos códigos indican que se deben tomar más acciones para completar la solicitud, generalmente redirigiendo a una URL diferente..

4XX

CÓDIGOS DE ERROR DE CLIENTE

Estos códigos indican que hubo un problema con la solicitud del cliente.

5XX

CÓDIGOS DE ERROR DE SERVIDOR

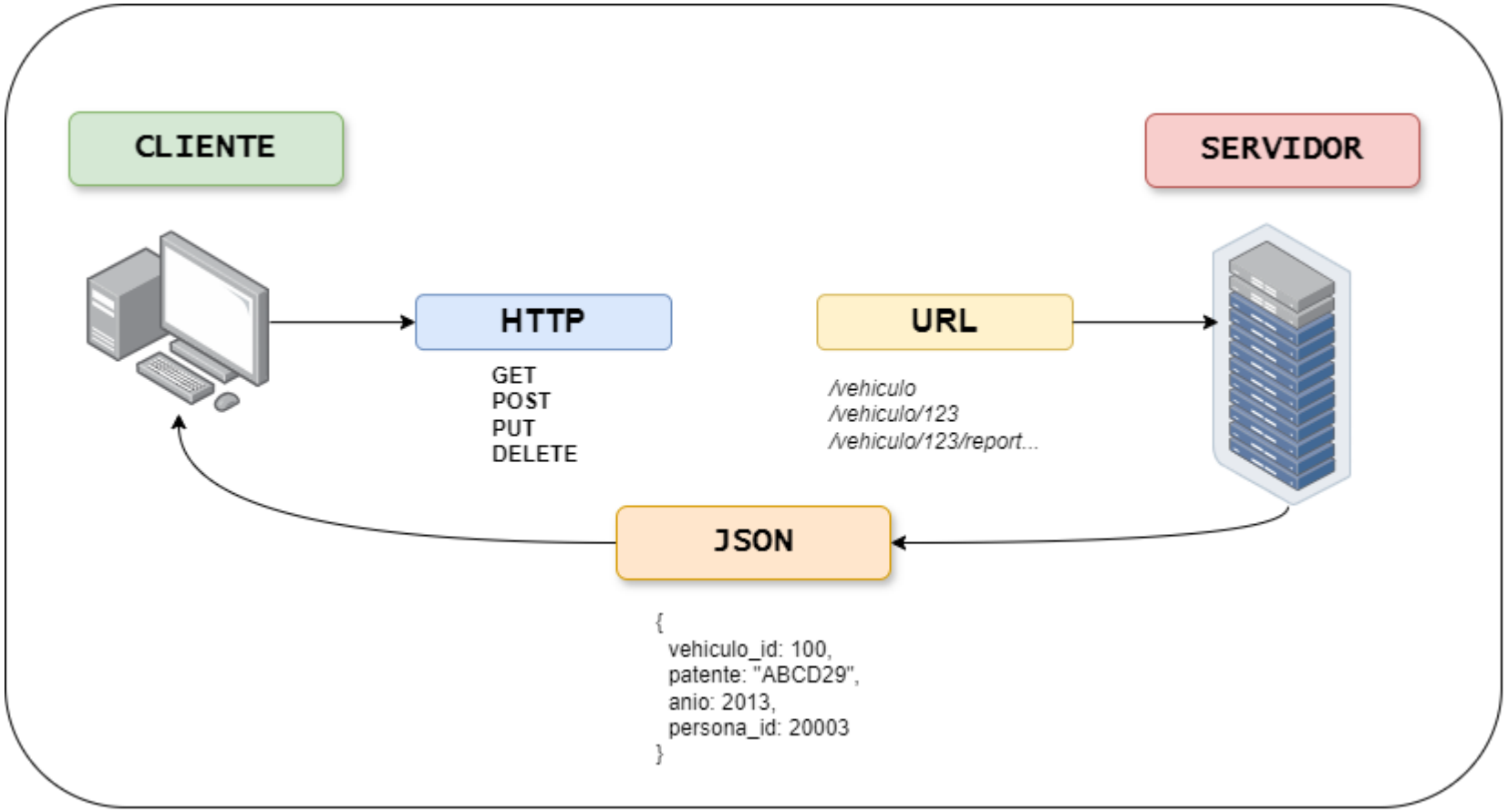
Estos códigos indican que el servidor falló al cumplir una solicitud válida.

- **500 Internal Server Error:** Se ha producido un error en el servidor y no se puede completar la solicitud.
- **501 Not Implemented:** El servidor no reconoce el método de solicitud y no puede soportarlo.
- **502 Bad Gateway:** El servidor recibió una respuesta inválida de un servidor upstream.
- **503 Service Unavailable:** El servidor no está disponible actualmente (sobrecargado o en mantenimiento).

03

REST

¿Qué se entiende por **REST**?



• REST

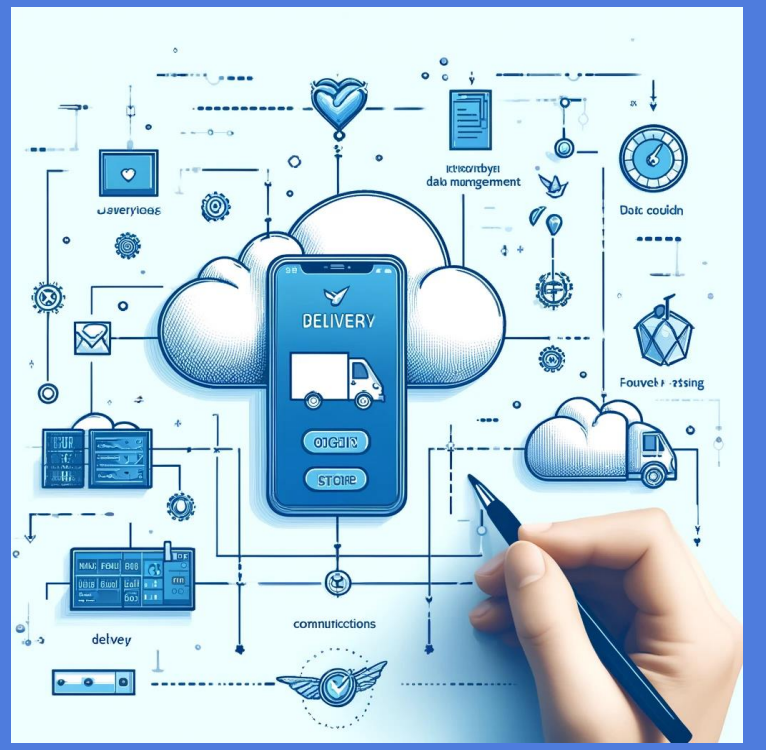
REST (Representational State Transfer) es una arquitectura de software que se utiliza para diseñar servicios web que permiten la comunicación entre diferentes sistemas a través de la red, especialmente en la web.

Es fundamental en el desarrollo de **APIs (Interfaces de Programación de Aplicaciones)** que permiten que diferentes aplicaciones se **comuniquen entre sí**.

REST utiliza los métodos HTTP (**GET, POST, PUT, PATCH, DELETE**) de manera específica para realizar operaciones CRUD (**Crear, Leer, Actualizar, Eliminar**) en los recursos.

Se centra en la manipulación de recursos a través de una **API web** utilizando métodos **HTTP** estándar.

Cada solicitud del cliente al servidor debe contener toda la información necesaria para entender y procesar la solicitud, ya que el **servidor no guarda ninguna información sobre el estado del cliente entre solicitudes**. Esto se conoce como **Stateless** (Sin estado).



• Ventajas de usar REST en el backend

SIMPLICIDAD Y ESTÁNDAR

Al utilizar métodos HTTP estándar, REST es fácil de entender y usar.

ESCALABILIDAD

La naturaleza stateless de REST facilita la escalabilidad, ya que cada solicitud es independiente.

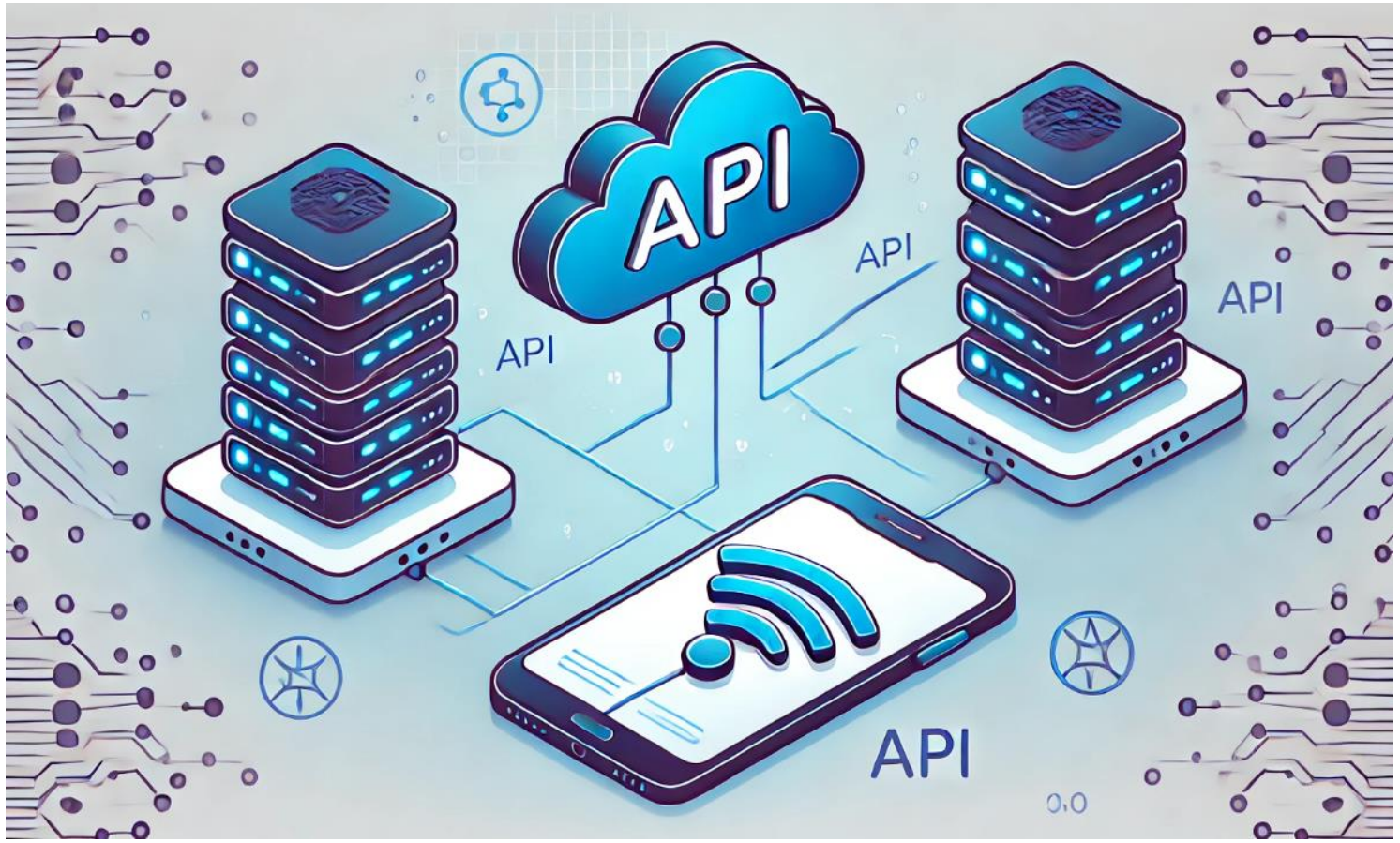
FLEXIBILIDAD

REST permite el uso de diferentes formatos de datos (como JSON, XML, HTML), lo que lo hace muy flexible.

INTEROPERABILIDAD

Al seguir estándares abiertos, las APIs RESTful pueden ser utilizadas por diferentes aplicaciones y sistemas, independientemente de la plataforma en la que estén desarrolladas.

¿Qué son las **APIs**?



• APIs

API (Application Programming Interface), es una interfaz que permite que **dos aplicaciones se comuniquen entre sí**. Las APIs definen el conjunto de reglas y protocolos que las aplicaciones deben seguir para solicitar y compartir datos o funcionalidades.

SOLICITUD

El cliente (la aplicación que solicita información) envía una solicitud a la API.

RESPUESTA

El servidor (la API) procesa la solicitud y devuelve una respuesta con los datos solicitados.

FORMATO DE DATOS

JSON, XML, CSV, YAML, etc.

• Formato de datos en APIs

JSON (JavaScript Object Notation)

Formato ligero y fácil de leer/escribir.

informacion.json

```
A > {} informacion.json > ...
1  {
2      "id": 123,
3      "run": "12345678-K",
4      "nombre": "Diego Palmeiro",
5      "correo": "d.palmeiro@duoc.cl",
6      "direccion": {
7          "calle": "Calle O'Higgins nº 123456",
8          "ciudad": "Concepción",
9          "pais": "Chile"
10     },
11     "telefonos": [
12         {
13             "tipo": "casa",
14             "numero": "123-456-789"
15         },
16         {
17             "tipo": "movil",
18             "numero": "987-654-321"
19         }
20     ]
21 }
```

XML (eXtensible Markup Language)

Formato más pesado y estructurado.

informacion.xml

```
A > informacion.xml
1  <usuario>
2      <id>123</id>
3      <run>12345678-K</run>
4      <nombre>Diego Palmeiro</nombre>
5      <correo>d.palmeiro@duoc.cl</correo>
6      <direccion>
7          <calle>Calle O'Higgins nº 123456</calle>
8          <ciudad>Concepción</ciudad>
9          <pais>Chile</pais>
10     </direccion>
11     <telefonos>
12         <telefono>
13             <tipo>casa</tipo>
14             <numero>123-456-789</numero>
15         </telefono>
16         <telefono>
17             <tipo>movil</tipo>
18             <numero>987-654-321</numero>
19         </telefono>
20     </telefonos>
21 </usuario>
```

• Buenas prácticas para el diseño de APIs REST

Una **API REST (Interfaz de Programación de Aplicaciones Representational State Transfer)** es un conjunto de reglas y convenciones para crear e interactuar con servicios web. **REST** es una **arquitectura** de software que impone ciertas restricciones para el diseño de los servicios web, lo que facilita la creación de sistemas escalables, mantenibles y eficientes.

- **SIMPLICIDAD**: Las APIs REST son fáciles de entender y usar debido al uso de HTTP y sus métodos estándar.
- **ESCALABILIDAD**: La arquitectura REST permite la construcción de sistemas escalables.
- **FLEXIBILIDAD**: Los recursos pueden ser representados en múltiples formatos.
- **INTEROPERABILIDAD**: Las APIs REST pueden ser consumidas por diferentes plataformas y lenguajes de programación.

• Buenas prácticas para el diseño de APIs REST

RECOMENDACIONES

Versionado de la API

Facilita la evolución de la API sin romper la compatibilidad con versiones anteriores.

```
/api/v1/libros

/api/v2.1/personas

/api/v2.2beta/boletas

/api/v2024/ventas
```

• Buenas prácticas para el diseño de APIs REST

RECOMENDACIONES

Nombres Descriptivos y Consistentes

Utiliza sustantivos en **plural** para los nombres de los recursos.

`/api/v1/productos` en lugar de **`/api/v1/producto`**

`/api/v1/estudiantes/11/notas` en lugar de **`/obtenerNotasEstudiante/11`**

`/api/v1/productos/11` **`/api/v1/boleta/2000`** **`/api/v1/compra/2000`**

• Buenas prácticas para el diseño de APIs REST

RECOMENDACIONES

Nomenclatura de API para Recursos con Nombres Múltiples Palabras

Cuando tienes nombres de recursos que consisten en más de una palabra, es importante mantener una convención de nombres clara y consistente. La convención más común es utilizar la notación **kebab-case** o **camelCase** para separar las palabras.

```
/api/v1/obtenerDatos
/api/v1/obtener-datos

/api/v1.2beta/informesDeVentas
/api/v1.2beta/informes-de-ventas
```

```
/api/v2/registrosAcademicos
/api/v2/registros-academicos

/api/v2025/informesDeVentas
/api/v2025/informes-de-ventas
```

• Buenas prácticas para el diseño de APIs REST

RECOMENDACIONES

Uso Correcto de Verbos HTTP

- GET para obtener recursos.
- POST para crear nuevos recursos.
- PUT para actualizar recursos existentes.
- DELETE para eliminar recursos.

GET

POST

PUT

DELETE

Usa los Códigos Correctos

- 200 OK para solicitudes exitosas.
- 201 Created para creación exitosa de recursos.
- 204 No Content para eliminaciones exitosas.
- 400 Bad Request para solicitudes inválidas.
- 401 Unauthorized para acceso no autorizado.
- 404 Not Found para recursos no encontrados.
- 500 Internal Server Error para errores del servidor.

• Buenas prácticas para el diseño de APIs REST

RECOMENDACIONES

Parámetros de Ruta y Consulta

- Utiliza parámetros de ruta para identificar recursos específicos.
- Utiliza parámetros de consulta para filtros y opciones.
- Ejemplo:
    - Parámetro de ruta: `/api/v1/products/{id}`
    - Parámetro de consulta: `/api/v1/products?category=electronics`

Jerarquía y Anidamiento Lógico

- Organiza los endpoints para reflejar relaciones entre recursos.
- `/api/v1/products/{productId}/reviews` para las reseñas de un producto específico.

• Buenas prácticas para el diseño de APIs REST

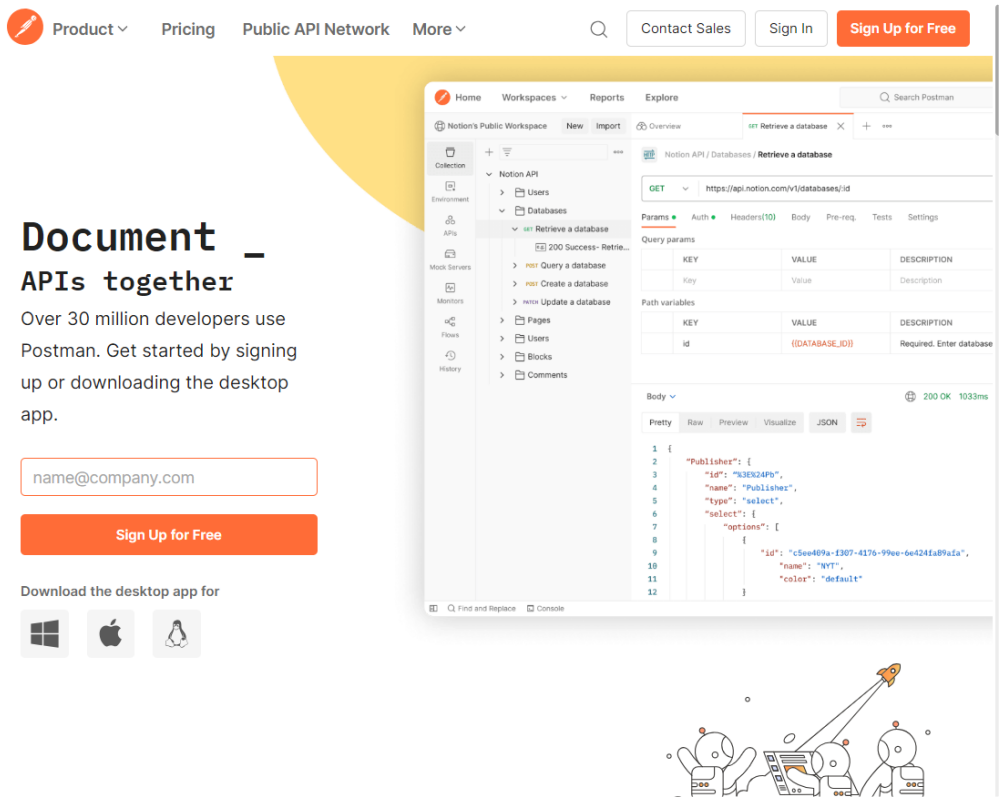
API	MÉTODO HTTP	PATH	CÓDIGO DE ESTADO	DESCRIPCIÓN
Listar libros	GET	/api/v1/libros	200 (OK)	Se recupera todos los recursos
Guardar libro	POST	/api/v1/libros	201 (Created)	Se crea un nuevo recurso
Obtener libro	GET	/api/v1/libros/{id}	200 (OK)	Se recupera un recurso
Actualizar libro	PUT	/api/v1/libros/{id}	200 (OK)	Se actualiza un recurso
Eliminar libro	DELETE	/api/v1/libros/{id}	204 (No Content)	Se elimina un recurso

• Buenas prácticas para el diseño de APIs REST

API	MÉTODO HTTP	PATH	CÓDIGO DE ESTADO	DESCRIPCIÓN
Listar prestamos	GET	/api/v1/prestamos	200 (OK)	Se recupera todos los recursos
Guardar prestamo	POST	/api/v1/prestamos	201 (Created)	Se crea un nuevo recurso
Obtener prestamo	GET	/api/v1/prestamos/{id}	200 (OK)	Se recupera un recurso
Actualizar prestamo	PUT	/api/v1/prestamos/{id}	200 (OK)	Se actualiza un recurso
Eliminar prestamo	DELETE	/api/v1/prestamos/{id}	204 (No Content)	Se elimina un recurso

• POSTMAN

Postman es una **herramienta** de desarrollo de software utilizada para **probar APIs.**



POSTMAN

Ver presentación rPruebas

IR A POSTMAN

04

Sintaxis Básica de Spring Boot

• Variables y Funciones

Python

```python
mensaje = "Hola"
version = 1

def saludar():
    return f"{mensaje} v{version}"
```

Java

```java
public class Main {
  private static String mensaje = "Hola";
  private static int version = 1;

  public static String saludar() { return mensaje + " v" + version; }
}
```

Spring Boot

```java
@RestController
public class ApiController {
  private String mensaje = "Hola";
  private int version = 1;

  @GetMapping("/saludo")
  public String saludar() { return mensaje + " v" + version; }
}
```

Diferencia clave: en Spring Boot tu función vive en un controlador y se expone por HTTP con **@GetMapping**

(atajo de @RequestMapping(method=GET))

¿Qué son las @anotaciones en Spring?

*"En Spring Boot tenemos algo llamado **anotaciones** y su característica principal es usar el simbolo **"@"** al frente de estas palabras claves. Las anotaciones realizan ciertos procesos internos que a su vez Spring Boot detecta y con uso ya sabe qué tipo de clase es. Si estas anotaciones no existieran entonces nosotros como **developers** tendríamos que especificar qué tipo de clase es, siendo esto mucho más complicado..."*

ESCUELA DE INFORMÁTICA Y TELECOMUNICACIONES

DuocUC®

• Estructuras de Control

Python

```python
# Ejemplo de estructuras de control en Python
edad = 20

# if / elif / else
if edad < 18:
    print("🚫 Menor de edad")
elif edad == 18:
    print("🟡 Justo en la mayoría de edad")
else:
    print("✅ Mayor de edad")

# switch simulado (Python usa 'match' desde 3.10)
opcion = "B"
match opcion:
    case "A":
        print("Elegiste la opción A")
    case "B":
        print("Elegiste la opción B")
    case _:
        print("Opción inválida")
```

Java

```java
public class EstructurasJava {
    public static void main(String[] args) {
        int edad = 20;

        // if / else if / else
        if (edad < 18) {
            System.out.println("🚫 Menor de edad");
        } else if (edad == 18) {
            System.out.println("🟡 Justo en la mayoría de edad");
        } else {
            System.out.println("✅ Mayor de edad");
        }

        // switch clásico
        String opcion = "B";
        switch (opcion) {
            case "A":
                System.out.println("Elegiste la opción A");
                break;
            case "B":
                System.out.println("Elegiste la opción B");
                break;
            default:
                System.out.println("Opción inválida");
        }
    }
}
```

• Estructuras de Control

Spring Boot

```java
@RestController
@RequestMapping("/api/v1")
public class ControlController {

    @GetMapping("/control")
    public String verificar(@RequestParam int edad, @RequestParam String opcion) {

        // if / else if / else
        String mensaje;
        if (edad < 18) {
            mensaje = "🚫 Menor de edad";
        } else if (edad == 18) {
            mensaje = "🟡 Justo en la mayoría de edad";
        } else {
            mensaje = "✅ Mayor de edad";
        }

        // switch moderno (Java 17+ soporta '->' simplificado)
        String resultado = switch (opcion) {
            case "A" -> "Elegiste la opción A";
            case "B" -> "Elegiste la opción B";
            default -> "Opción inválida";
        };

        return mensaje + " | " + resultado;
    }
}
```

• Ciclos

Python

```python
# Ejemplo de ciclos en Python
productos = ["Teclado", "Mouse", "Pantalla"]

# for: recorrer una lista
for p in productos:
    print("📦", p)

# while: repetir hasta que se cumpla una condición
contador = 0
while contador < 3:
    print("🔢 Número:", contador)
    contador += 1

print("✅ Ciclos completados")
```

Java

```java
import java.util.*;

public class CiclosJava {
    public static void main(String[] args) {
        List<String> productos = Arrays.asList("Teclado", "Mouse", "Pantalla");

        // for: recorrer lista
        for (String p : productos) {
            System.out.println("📦 " + p);
        }

        // while: repetir hasta que se cumpla condición
        int contador = 0;
        while (contador < 3) {
            System.out.println("🔢 Número: " + contador);
            contador++;
        }

        System.out.println("✅ Ciclos completados");
    }
}
```

• Ciclos

Spring Boot

```java
@RestController
@RequestMapping("/api/v1")
public class CiclosController {

    @GetMapping("/ciclos")
    public Map<String, Object> mostrarCiclos() {

        // Lista de productos
        List<String> productos = Arrays.asList("Teclado", "Mouse", "Pantalla");
        List<String> salida = new ArrayList<>();

        // for: recorrer lista
        for (String p : productos) {
            salida.add("📦 " + p);
        }

        // while: repetir tres veces
        int contador = 0;
        List<Integer> numeros = new ArrayList<>();
        while (contador < 3) {
            numeros.add(contador);
            contador++;
        }

        // Retornar como JSON
        Map<String, Object> resultado = new HashMap<>();
        resultado.put("productos", salida);
        resultado.put("numeros", numeros);
        resultado.put("mensaje", "✅ Ciclos completados");

        return resultado;
    }
}
```

• GUÍA PRÁCTICA 1

Ingresa al AVA de esta actividad y desarrolla la guía práctica

1.1.5 Actividad 1 - Mi primer proyecto en Spring Boot

No olvides aplicar control de versiones. Aprende cómo en

1.1.6 Guía control de versiones Git y Github

DuocUC®

CERCANÍA. LIDERAZGO. FUTURO.

duoc.cl

7 AÑOS ACREDITADO

Comisión Nacional de Acreditación CNA-Chile

DESDE AGOSTO 2017 HASTA AGOSTO 2024.
DOCENCIA DE PREGRADO. GESTIÓN INSTITUCIONAL. VINCULACIÓN CON EL MEDIO.